Designing an AI video pipeline that Claude Code can run: the LLM chooses, the code computes

2026-10-13

I wanted a repeatable way to turn a topic and a few sources into a three-minute narrated explainer video. My first plan was a one-page sketch: write a script with an LLM, read it aloud with text-to-speech, transcribe the audio for timestamps, ask an LLM to turn the timestamps into a storyboard, generate the pictures, and assemble the video. The shape was right. When I reviewed it critically, though, several details were wrong or could not be built as written. This post is the corrected design, and the reasoning behind each correction.

1 The idea in one line

The LLM chooses; the code computes. Claude decides what the script says, where the scenes cut and whether the pictures look right. Python does everything with a right answer: speech synthesis, timing arithmetic, cost estimates and rendering. Almost every fix below is an application of that split.

2 What was wrong with the first sketch

Problem in the sketch	What I changed
"3 minutes is about 600–750 words"	That is 200–250 words per minute, far faster than speech. At about 155 wpm, 3 minutes is roughly 465 words. (The figure looks like a Chinese character count carried over to English.)
The LLM was asked to write timestamps with "no gaps"	Models break that constraint silently. The LLM now returns only cut points as word indices, and code derives the times
Whisper transcribes the audio to get timings	I already know the script. Forced alignment against the known text avoids mis-transcriptions
Word timestamps requested, then saved as `.srt`	An SRT keeps only segment-level times. The word-level `words.json` is the master clock, and SRT is a derived export
The LLM also emitted a combined voiceover string	Duplicated text can drift from the sections. Code joins the sections instead
Midjourney through a script	There is no official API, so scripted use depends on unofficial wrappers
No approval step	Nothing stopped a bad script from reaching paid image and video generation

3 The pipeline

```
brief → script → fact-check → [approve script]
      → speech → align → beatsheet → validate
      → cost estimate → [approve spend]
      → assets → visual QA → render → final QA and packaging
```

Audio is the master clock. The script is synthesised first, per section, so section boundaries come for free and one bad section can be redone alone. Forced alignment then produces `words.json`, with a start and end time for every word. Every visual is anchored to those times.

The beatsheet is word-indexed. The director step sees the script and a

transcript numbered by word (`0: Why`, `1: do`, ...), and returns scenes like this:

```
{"scene": 1, "first_word": 0, "last_word": 8,
 "visual_type": "still", "visual_prompt": "Close-up of a confused cartoon charac
 "motion_prompt": "slow push-in", "on_screen_text": null}
```

A small validator then does the arithmetic. It checks that scenes cover every word with no gaps, converts word indices to times, snaps to the 30 fps frame grid, enforces 2–5 seconds per scene, and confirms the total matches the audio length. If it fails, the error goes back to Claude once; a second failure stops the run. The style words ("flat vector illustration") live only in a style file and are appended at generation time, so they cannot conflict with the prompts.

Cost-shaped visuals. A three-minute video at three seconds a scene is about 60 scenes. Image-to-video on all of them would be expensive and slow, so most scenes are a still with a camera move, and a small capped number are real video clips. Scenes that are really diagrams or text are not generated at all.

4 Two gates before any money is spent

1. **Approve the script.** Claude's fact-checker compares each claim in the script with the sources and flags the unsupported ones. I read the script before any speech is synthesised.
2. **Approve the spend.** A script counts the images, video seconds and speech characters, multiplies by a price file, and writes an estimate. If it exceeds the cap it fails. Only my approval creates the marker file that lets the paid scripts run.

A hook enforces this, and I like the reasoning for it: a guard earns its place by naming the specific irreversible harm it prevents. Here the harm is paid API calls made without approval or over budget. A second hook validates every JSON artefact against its schema as it is written, so a malformed file cannot reach a paid stage. Editing the script after the first gate invalidates everything downstream and removes the approvals.

5 How it maps onto Claude Code

- **Skills.** The two prompts from the first sketch become real `SKILL.md` files: a script-writer and a beat-director, each pointing at its JSON schema.
- **Subagents.** A read-only fact-checker, and a visual-QA agent. The QA agent gets a contact sheet (one labelled frame per scene, built with ffmpeg) and checks style consistency, legibility and whether each picture matches its prompt. Claude can read images, so this works. Bad scenes are regenerated individually, at most twice, and then they come to me.
- **Plan mode** for the two gates, and a few slash commands to scaffold a job, resume it, and print its status and spend.
- **A manifest** records, per stage, the input hash and the output, plus seeds and per-scene costs. Re-running skips anything unchanged, which matters because image and video APIs are asynchronous and sometimes fail halfway.

The pipeline lives in its own repository. It is not a good fit for this site's repository, whose rules deliberately forbid adding build machinery.

6 The five decisions, and why

Decision	Choice	Why
Format	16:9, 3 minutes, developed against a 30–60 second cut	Failed runs are expensive at 60 scenes, so test short. Vertical video is a second pass, not a setting: captions, framing and prompts all change
Language	English	The tools I want all work well in English
Speech	edge-tts while developing; ElevenLabs or Fish Audio to publish, behind one interface	edge-tts is free and returns word timings, but it uses an unofficial endpoint with unclear commercial terms

Decision	Choice	Why
Images	Flux through a hosted API, behind one function	Local Flux on a laptop is slow for batches of 60. Image models garble text, so text and diagram scenes are rendered by the compositor
Assembly	Remotion for compositing, ffmpeg for audio and checks	Animated captions, transitions and text or UI scenes are real motion graphics. In ffmpeg they become fragile filter graphs; in Remotion they are components reading the compiled JSON

I changed my mind on the last one. My first draft defaulted to ffmpeg, then I noticed that its own storyboard had visual types ffmpeg handles badly. Remotion has a licence that is free for individuals and small teams and paid for larger companies, so I will read the terms against my situation before publishing anything.

7 What I would still check

- **Rights.** Stock voices only, no cloning of real people. Record the licence of every music, effect and stock-footage asset in the manifest.
- **Disclosure.** Add the platform's AI-content label and put the line in the description.
- **Drift.** If a speech or image model version changes, outputs are not guaranteed to reproduce. The manifest records versions so I notice.
- **Build order.** Schemas and the validator first, since they are pure code and cost nothing. Then speech and alignment on a 30-second sample, the Claude stages as a dry run that ends at the cost estimate, and only then image generation and the Remotion scenes, one component at a time.

The definition of done is one full video produced from a brief with exactly

two human approvals, no hand-edited timestamps, a resumable manifest, and actual spend within 10% of the estimate. When I have built it I will write a follow-up with what broke.